

PRÁCTICA 2: PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Juan Sebastian Bonces Junco - 2234562
Santiago Benitez Torres - 221752
Integrante3 - código3

https://github.com/Elbonces/2026_2_CommII_A1_GA11

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones

Universidad Industrial de Santander

31 de agosto de 2026

Abstract—Completar

Index Terms—GNU Radio, Embedded Python Blocks, Accumulator, Differentiator.

I INTRODUCCIÓN

Completar[1].

II METODOLOGÍA

El propósito de la práctica consistió en implementar cuatro propuestas de diseño, fundamentadas en el desarrollo de bloques funcionales y en la validación estadística de las señales.

II-A Diseño e Implementación de bloques

Para el desarrollo de la práctica en GNU Radio, se utilizaron bloques de Python embebidos (*Embedded Python Blocks*), los cuales permitieron programar la lógica interna requerida para la implementación de:

- **Acumulador:** Corresponde al algoritmo diseñado para sumar de forma acumulativa las muestras de entrada, facilitando los procesos de recuperación de señales:

$$y[n] = \sum_{k=0}^n x[k] \quad (1)$$

- **Diferenciador:** Corresponde al algoritmo que resalta los cambios o transiciones rápidas de amplitud a través de la diferencia entre muestras sucesivas:

$$y[n] = x[n] - x[n - 1] \quad (2)$$

- **Operadores Estadísticos:** Se implementaron algoritmos orientados a la extracción en tiempo real de

diversas métricas, tales como el valor eficaz (RMS), la media ($\bar{x}$), la desviación (σ_x) y la potencia (P).

III RESULTADOS

Se estudió el funcionamiento de los bloques dentro de GNU Radio a través de la aplicación de algoritmos de diferenciación, acumulación y evaluación estadística.

III-A Acumulador

III-A1. Señal de entrada cuadrada

El acumulador actúa como integrador en tiempo discreto, suavizando cambios bruscos y compensando ruido al sumar muestras. Se evaluó con una señal cuadrada, 32 kHz, un Vector Source de 8 muestras y un Throttle. La salida obtenida tuvo forma de rampa, confirmando el correcto funcionamiento del bloque como integrador.

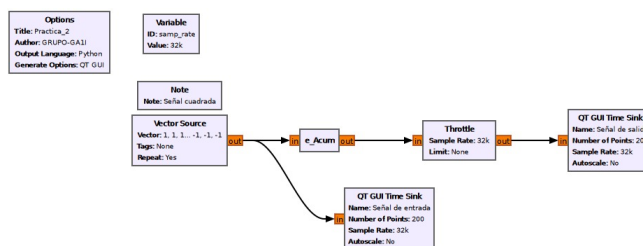


Figura 1: Diagrama de bloques del acumulador

Análisis de señal: La obtención de una salida con forma de rampa evidencia el correcto funcionamiento del bloque como un integrador en tiempo discreto.

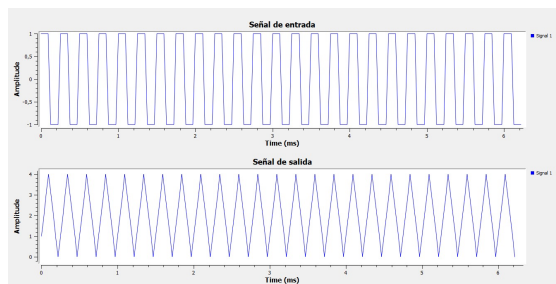


Figura 2: Señal de entrada y salida

III-A2. Señal de entrada cuadrada, variando el número de muestras.

Se duplicó el número de muestras de 8 a 16, manteniendo 32 kHz. La salida conservó la forma de rampa, con el doble de amplitud, confirmando que el acumulador es recursivo y que su estado interno depende del número de muestras con el mismo signo, no de la amplitud de entrada.

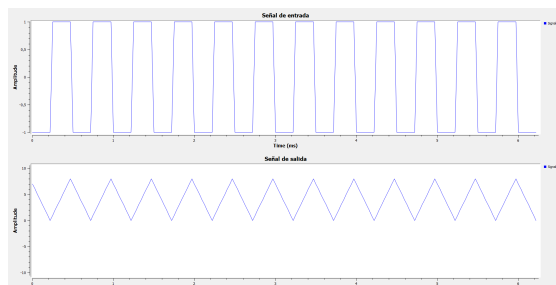


Figura 3: Señal de entrada y salida del acumulador con 16 muestras.

III-A3. Señal de entrada cuadrada, variando la frecuencia de muestreo.

Se redujo la frecuencia de muestreo de 32 kHz a 2 kHz, sin cambiar el Vector Source. La salida no cambió en forma ni amplitud, confirmando la dependencia del número de muestras, aunque cada ciclo tardó más en completarse, evidenciando menor resolución temporal.

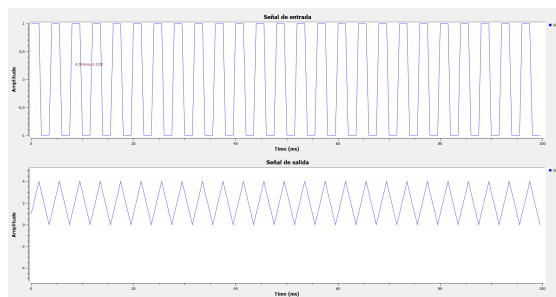


Figura 4: Señal de entrada y salida del acumulador con frecuencia de muestreo de 2 kHz.

III-B Diferenciador

Corresponde a un sistema diseñado para procesar la señal de entrada y resaltar las transiciones o cambios rápidos en su amplitud

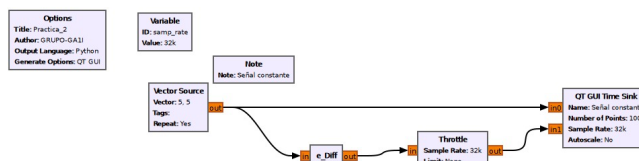


Figura 5: Diagrama de bloques del diferenciador

III-B1. Diferenciador con entrada constante

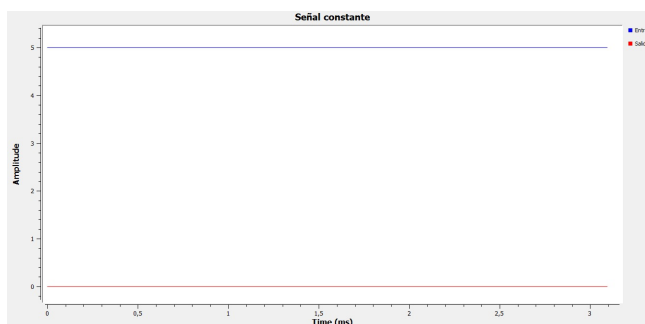


Figura 6: Respuesta ante una entrada constante

Como se observa en la gráfica obtenida, la señal de salida toma únicamente valores de cero. Esto ocurre porque, al no registrarse variaciones en la amplitud de entrada, el bloque efectúa de forma constante la resta entre la muestra actual y la anterior (5-5), confirmando que el diferenciador responde exclusivamente ante cambios en la señal.

III-B2. Diferenciador con entrada rampa

Nuestra señal consta de una entrada en forma de rampa con magnitud máxima de 6 y con pendiente 1.

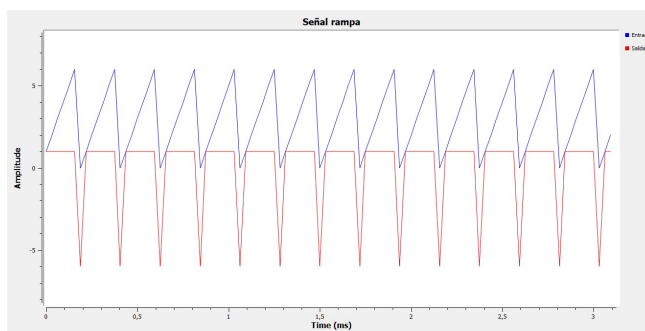


Figura 7: Respuesta ante una entrada rampa

Con una entrada tipo rampa (Fig. 7), la salida se estabiliza en un valor constante de 1, debido a la pendiente unitaria fija entre muestras. Al alcanzar el valor máximo (6) y reiniciarse a 0, el diferenciador genera un pico de amplitud -6 , al restar el valor mínimo del máximo.

III-B3. Diferenciador con entrada cuadrada

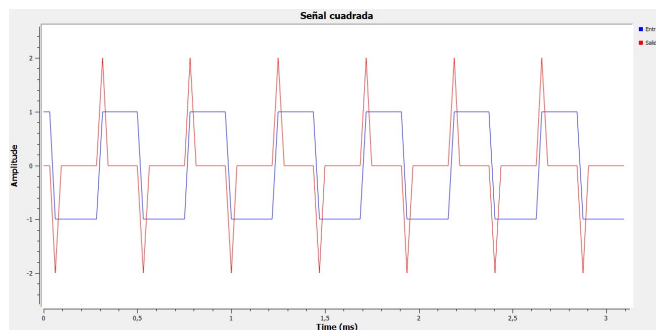


Figura 8: Respuesta ante una entrada cuadrada

Pudimos notar en la Fig. 10 que en la salida aparecieron picos claros en las transiciones de la señal: uno positivo al subir y uno negativo al bajar. Estos picos reflejan el cambio abrupto entre muestras consecutivas, permitiendo identificar exactamente los flancos de subida y bajada de la señal de entrada.

III-C Análisis Estadístico en Tiempo Real.

Se implementó un diagrama de flujo mediante un Embedded Python Block que calcula en tiempo real la media, media cuadrática, RMS, potencia promedio y desviación estándar de una señal de entrada, usando sumas acumulativas (cumsum) que actualizan cada métrica muestra a muestra sin necesidad de almacenar el historial completo de la señal.

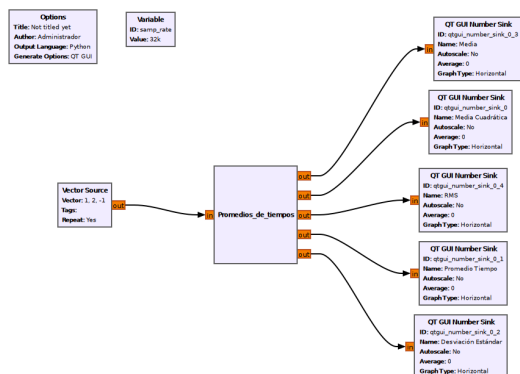


Figura 9: Diagrama de flujo del bloque de análisis estadístico.

Para la validación inicial se usó un Vector Source con el vector (1, 2, -1) a 32 kHz. Los valores obtenidos (Media = 0.6667, Media Cuadrática = 2.0, RMS = 1.4142, Potencia promedio = 2.0, Desviación Estándar = 1.2472) coincidieron con los teóricos, validando el bloque. La potencia promedio y la media cuadrática arrojaron el mismo valor, consistente con la teoría, ya que para una señal real la potencia promedio equivale al valor cuadrático medio.

Validación bajo ruido

Se construyó un segundo diagrama de flujo en el cual una señal cuadrada de amplitud ± 1 se suma, mediante un bloque Add, con un Noise Source gaussiano, antes de ingresar al analizador estadístico, con el fin de evaluar su robustez ante señales contaminadas. Se comparó el comportamiento del sistema con amplitud de ruido 0 y 0.1.

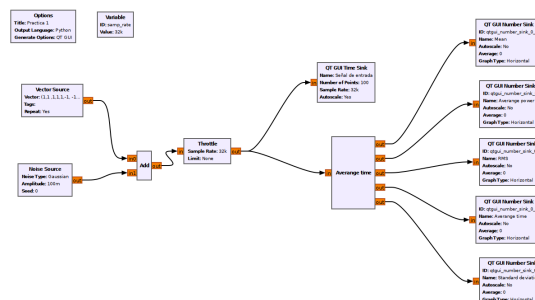


Figura 10: Diagrama de flujo para la validación del analizador estadístico bajo ruido gaussiano.

Tabla 1: Validación de bloques: teórico vs. ejecución bajo ruido.

Métrica	Ruido = 0	Ruido = 0.1
Media (teo.)	0	≈ 0
Media (obt.)	0.000244	0.193174
RMS (teo.)	1	$\approx 1,005$
RMS (obt.)	1.000000	1.007665
Potencia (teo.)	1	$\approx 1,01$
Potencia (obt.)	1.000000	1.015388
Desv. Est. (teo.)	1	$\approx 1,005$
Desv. Est. (obt.)	1.000000	0.988975

Con ruido de amplitud 0, los valores coincidieron con los teóricos para una señal cuadrada de amplitud unitaria. Con ruido de amplitud 0.1, el RMS, la potencia promedio y la desviación estándar se mantuvieron cercanos a lo esperado, pero la media osciló sin estabilizarse, debido a errores de redondeo en la suma acumulada en precisión float32.

Tabla 2: Resultados de bloques estadísticos con variaciones de vector, Fs y N.

Nº Muest.	Fs [kHz]	RMS	Media	Potencia Prom.	Desv. Est.
3	32	1.4142	0.6667	2.0000	1.2472
6	32	1.4142	0.6667	2.0000	1.2472
3	2	1.4142	0.6667	2.0000	1.2472
3	320	1.4142	0.6667	2.0000	1.2472
3	32	2.8285	1.3333	8.0003	2.4945

Ni la duplicación de muestras ni la variación de la frecuencia de muestreo alteraron los valores estadísticos, lo que confirma que estos dependen únicamente del vector de entrada. Al escalar el vector al doble (2,4,−2), la media y el RMS se duplicaron, mientras que la potencia promedio y la media cuadrática se cuadruplicaron (pasando de 2,0 a 8,0), validando la sensibilidad del bloque ante cambios en la energía de la señal.

III-D Aplicación propuesta: Detección de degradación de señal por ruido en un enlace de comunicación.

Un problema común en los sistemas de comunicaciones es la degradación de la señal transmitida debido al ruido del canal. El analizador estadístico desarrollado permite monitorear en tiempo real la potencia promedio y el valor RMS de la señal recibida, funcionando como un indicador simplificado de la relación señal a ruido (SNR).

Para evaluar esta aplicación, se reutilizó el diagrama de flujo de validación mediante la adición de un bloque Noise Source a la señal deseada. El sistema se ejecutó bajo tres escenarios de canal:

1. **Canal limpio:** amplitud de ruido 0.
2. **Ruido moderado:** amplitud de ruido 0,1.
3. **Ruido severo:** amplitud de ruido 0,5.

Al incrementar la intensidad del ruido, se observa un aumento proporcional en el RMS y la potencia promedio medida. Este comportamiento responde a que la energía total del sistema corresponde a la suma de las contribuciones de la señal de información y del ruido introducido por el canal.

Los resultados demuestran la utilidad del bloque estadístico como un sistema de detección y monitoreo en tiempo real: al establecer un umbral límite basado en los parámetros de la señal limpia, cualquier incremento inusual en la potencia o el RMS permite alertar de manera automática sobre la degradación del canal de comunicación.

IV CONCLUSIONES

- Se demostró que la implementación de bloques personalizados en Python (Embedded Python

Blocks) permite ampliar significativamente las capacidades nativas del entorno de radio definida por software, posibilitando el diseño de algoritmos propios de procesamiento de señales —como acumulación, diferenciación y análisis estadístico— y su validación experimental frente a los resultados teóricos esperados. Esto evidencia que dichas herramientas no solo sirven para interconectar bloques predefinidos, sino también para desarrollar funcionalidades específicas adaptadas a necesidades particulares de análisis de señales en tiempo real

Referencias

- [1] A. V. Oppenheim y R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.
- [2] GNU Radio Project, “GNU Radio Documentation.” [Online]. Available: <https://www.gnuradio.org>
- [3] P. Z. Peebles, *Probability, Random Variables, and Random Signal Principles*, 4th ed. McGraw-Hill, 2001.